

Trading Strategy

202566237 / 202553206

2026-02-19

Introduction

In 1755 Homma Munehisa published his work “The Fountain of Gold - The Three Monkey Record of Money” which explained his approach to trading rice futures on the Dojima Rice Exchange. He stated that market sentiment is the critical driver of price action, and explained his system of price analysis now called Candlestick Charting, one of the basic techniques of Technical Analysis - the discipline of inferring future price action from its history.

According to orthodox financial theory, Technical Analysis should not out-perform the market. Historical price information is available to the overwhelming majority of market participants, both amateur and professional, and so if any opportunity were to be gained from its analysis, it would be acted upon and disappear very quickly. This effect is one of several inferences drawn from the Efficient Market Hypothesis, the dominant theory of information-driven price action in modern finance. To the Efficient Market Hypothesis (EMH), technical analysis is little better than astrology [^2].

However, this conclusion is not universally accepted, and the EMH has been coming under increased scrutiny since the early 1980s. Whilst it is certainly a good model of what it means for a market to be efficient, and provides strong grounds for believing they should be, it does not follow that markets are efficient. For one thing, perfectly efficient markets are themselves theoretically impossible [^3]. Secondly, several studies of major indices indicate that even deep markets can exhibit price autocorrelation, particularly in times of market stress [^4]. Thirdly, the performance of Renaissance Technologies’ Medallion fund cannot be dismissed as just an anomaly [^5]. Indeed, it has been suggested that the acceptance of the universal application of the Efficient Market Hypothesis is directly correlated to its fashionability [^6].

Technical Indicators

Modern technical analysis has many techniques, ranging from subjective pattern-recognition, to high-frequency, fully-automated mathematical models. At the more objective but still easily applied end of this spectrum lie a group of models called “technical indicators”. These are popular with amateur traders because they are easy to understand and apply. Indeed, many of the most popular indicators were developed before high-power computing was widely available and were favoured because they are easy to compute. In fact, most technical indicators are types of signal estimation filters used in signal processing [^7]. Because they are all variations on the same basic mathematical structure, they tend to correlate closely and fall into four key categories [^8]:

1. *Momentum indicators*, attempt to measure the speed of price movements.
2. *Trend indicators*, attempt to detect the current trend and its state.
3. *Volatility indicators*, indicate changes in trading activity and uncertainty.
4. *Volume indicators*, measure market participation. e.g. A change in price associated with high volume indicates a widespread conviction in the new regime.

Consequently, they are usually used in combination with each other and other techniques. In fact, indicators are often composed of other indicators. For example, the Exponential Moving Average is the basis of the DEMA, ZLEMA, TRIX, and ATR indicators.

Given the above, our question is whether or not there exists an optimal combination of indicators that can be profitably applied without subjective intervention.

To answer this, we utilised a Genetic Programming strategy to evolve and test different combinations of indicators. This was chosen for two reasons: firstly, because it can provide easily understood answers, and secondly because evolution is an adaptive system, which correlates well with concept of performing in a constantly changing environment like a financial market.

ChatGPT was utilised for research, code analysis, and debugging, and some incidental code, as marked.

[^1] see, e.g. Hillier, D. (2024). eBook: Corporate Finance 5e. McGraw Hill

[^2] see Lo, A.W., Mamaysky, H. and Wang, J. (2000). Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation. *The Journal of Finance*, 55(4), pp.1705–1765

[^3] Grossman, S. J. and Stiglitz, J. E. (1980) ‘On the Impossibility of Informationally Efficient Markets’, *The American Economic Review*, 70(3), pp. 393–408

[^4] see, e.g. Rosini, L. and Shenai, V. (2020). Stock returns and calendar anomalies on the London Stock Exchange in the dynamic perspective of the Adaptive Market Hypothesis: A study of FTSE100 & FTSE250 indices over a ten year period. *Quantitative Finance and Economics*, 4(1), pp.121–147.

[^5] Cornell, B. (2020). Medallion Fund: The Ultimate Counterexample? *The Journal of Portfolio Management*, 46(4), pp.156–159.

[^6] Gyamfi, E.N., Kyei, K.A. and Gill, R. (2017). Market efficiency of African stock markets: A Meta-Analysis. *The Journal of Developing Areas*, 51(4), pp.69–80.

[^7] See Lo et al. (ibid) and Oppenheim, A.V. and Verghese, G.C. (2017). *Signals, systems and inference*. Harlow Pearson Education Limited.

[^8] portaladmin (2025). Axis Direct. [online] Axisdirect.in. Available at: <https://simplehai.axisdirect.in/117-good-reads-ri/8935-series-22-types-of-technical-indicators> Accessed 7 Apr. 2026.

Data

Three instruments were chosen for analysis: Berkshire-Hathaway, Apple, and Brent-Crude. The former two are equities, which exhibit a general upward trend, and the latter is a commodity that fluctuates within a range. They were selected because they are actively traded instruments with differing returns and volatilities, representing different market regimes. Data encompassing 2000 to 2025 was obtained, including several regime shifts and significant market events.

Trading Strategy

A context-free grammar of indicators from the TTR package for optimisation in a Genetic Program was created. The indicators were selected in order to ensure that all four of the above categories of indicator were represented.

```
##### Technical Analysis Indicators #####
# To guard against R's idiosyncratic tendency to allow inappropriate parameters
# or meaningless values such as NULL to silently poison the GP process, indicators
# return either a valid series or an empty one that can be handled by other (particularly
# the comp) function.

mk_guarded_indic <- \(f) {
  function(data) {
    tryCatch(f(data), error = \(e) return(data[0]))
  }
}
```

```

}
}

make_indicator_rules <- \(price_rules, volume, high_low_close) {
  list(expr = grule(compare(aspect, signal),
                           compare(const, signal),
                           compare(signal, signal)
                           ),
        compare = grule(comp),
        signal = grule(op(signal, indic),
                        op(signal, aspect),
                        indic
                        ),
        op = grule('+', '-', '*', '/'),
        indic = grule(sma(aspect, lag), # Trend; Simple MA
                      ema(aspect, lag), # Trend; Exponential MA
                      hma(aspect, lag), # Trend; Hull MA
                      vhf(aspect, lag), # Trend; Vertical Horizontal Filter
                      dema(aspect, lag, ratio, bool), # Trend; Double EMA
                      zlema(aspect, lag), # Trend; Zero Lag EMA
                      evwma(aspect, vol, lag), # Trend; Elastic Volume-Weighted MA
                      vwap(aspect, vol, lag), #Volume; Volume Weighted Average Price
                      cmf(hlc, vol, lag), #Volume; Chaikin Money Flow
                      cmo(aspect, lag), #Momentum; Chaikin Momentum Oscillator
                      rsi(aspect, lag, maType), #Momentum; Relative Strength Index
                      macd(aspect, lag, lag, lag, maType), #Mometum MACD
                      clv(hlc) # Hard to categorise; "Pressure"
                      ),
        aspect = do.call(grule, price_rules),
        ratio = gvrule(seq(0.1, 1.0, by = 0.1)),
        # These are commonly used window sizes in technical analysis
        lag = gvrule(c(2L, 3L, 5L, 10L, 14L, 20L, 50L, 100L, 200L)),
        const = gvrule(1:50),
        bool = grule(TRUE, FALSE),
        vol = do.call(grule, list(as.symbol(volume))),
        hlc = do.call(grule, list(high_low_close)),
        # Can't use WMA here
        maType = grule("EMA", "SMA", "DEMA", "ZLEMA", "HMA")
        )
}

```

The grammar is designed to create indicators that produce a signal, given context by comparison with either an aspect of the price series (the Open, High, Low, or Close), a constant, or another signal. The signal is converted into a time series of 1 or -1 if it is above or below the corresponding value in the left hand of the comparison.

The cost function (`assess_strat`) assumes that the account will hold a long position whilst the signal is 1, a short position if the signal is -1, and none if it is 0. It seeks to select strategies that outperform buy-and-hold. However, it tends to select very low frequency trading strategies that randomly select advantageous entry points in the training set, but fail to trade in the test set. Consequently, a threshold number of trades is set as a daily rate. In order to make the simulation more realistic and to penalise over-trading, the cost function adds a 0.1% charge for each trade. This reflects the fact that costs are very likely to be incurred either as a topline charge or in the spread, but are still usually very small. The final addition is a cost for inaccuracy.

```

target_trade_rate <- 1/21
# Cost function
assess_strat <- \(result, data) {
  stopifnot(is.xts(result) && is.xts(data))
  stopifnot(NROW(result) == NROW(data))
  next_day      <- stats::lag(data, -1) |> na.omit()
  idx           <- index(result)
  next_day      <- next_day[idx] # Force the training data to align with result
  actual_returns <- diff(log(next_day)) |> na.omit()
  actual_trend  <- sign(actual_returns)
  forecast      <- sign(result) # Just in case we need to preprocess result in future versions
  misses        <- sum(forecast != actual_trend)
  n_trades      <- sum(diff(forecast) != 0, na.rm = TRUE)
  # Remove low-trading strategies
  if (n_trades <= (NROW(data) * target_trade_rate)) return(Inf)
  buy_and_hold  <- sum(actual_returns)
  forecast      <- forecast[index(actual_returns)]
  return_path    <- as.numeric(forecast) * as.numeric(actual_returns)
  strat_return   <- sum(return_path, na.rm = TRUE)
  trade_rate     <- n_trades / length(result)
  trade_penalty  <- ((trade_rate - target_trade_rate) / target_trade_rate)^2
  fee            <- 0.01
  cost           <- (buy_and_hold - strat_return) + n_trades * fee + trade_penalty + misses/NROW(actual_returns)
  if (!is.finite(cost) || is.na(cost)) {
    Inf
  } else {
    cost
  }
}

```

GP Parameters

If `assess_strat` returns a value greater than the termination cost, the GP halts at that point. But since `assess_strat` reflects a strategy's relative out performance over buy and hold, we do not wish to cap this value. Consequently, the termination cost is unlimited.

```

max_iters <- 1000
#Cache the GP to save editing time (ChatGPT)
main_ga_path <- file.path(rds_dir, "brk_b_main_ga.rds")
main_ga <- load_or_run_rds(main_ga_path, function() {
  monitor <-<- monitor[0, ]
  ge <- GrammaticalEvolution(forecasting_grammar,
                             indicator_fit,
                             terminationCost = -Inf,
                             monitorFunc = monitor_func,
                             iterations = max_iters,
                             max.depth = 10)
  list(ge = ge, monitor = monitor)
})
ge <- main_ga$ge
monitor <- main_ga$monitor
plot(
  monitor$iteration,
  monitor$best_cost,

```

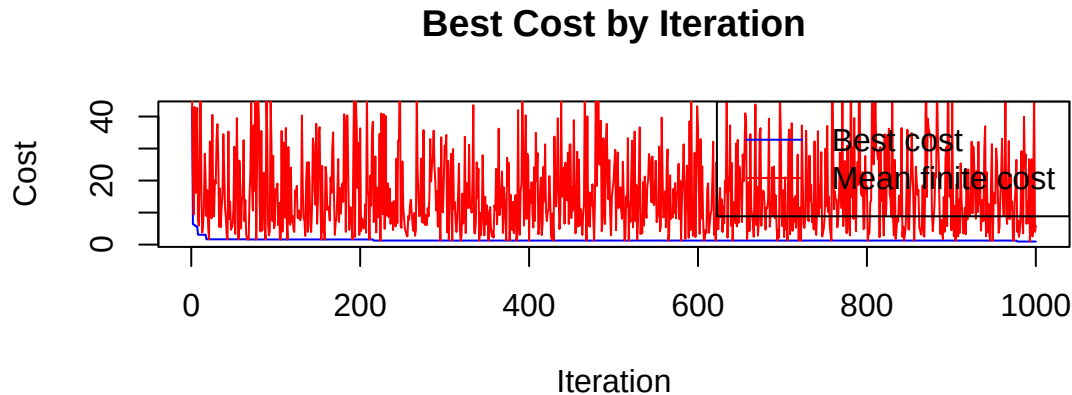
```

type = "l",
xlab = "Iteration",
ylab = "Cost",
main = "Best Cost by Iteration",
col = "blue"
)

lines(
  monitor$iteration,
  monitor$finite_mean,
  col = "red"
)

legend(
  "topright",
  legend = c("Best cost", "Mean finite cost"),
  col = c("blue", "red"),
  lty = 1
)

```



```

# Converts a strategy to a printable string
e_to_s <- \(ex) paste(deparse(ex), collapse = " ")

```

The above figure shows that the evolution progressed in steps, as a new elite function was discovered and remained in top position for extended periods whilst new generations failed to exceed its performance. It also shows that the solution was found very quickly, which might suggest that the grammar is too narrow.

One might suspect that the GA is static during these periods, but the red line shows that the mean cost per generation fluctuates significantly, indicating that the GA is at work.

Results

The strategy's performance was assessed by comparison to a buy and hold strategy and a permutation test to assess how well the strategy captured structurally important features of the asset's behaviour.

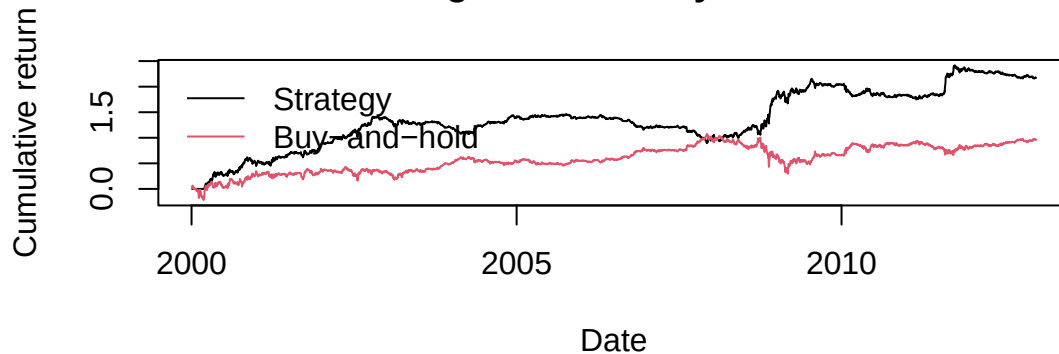
The permutation test shuffles the asset to create 1000 random return paths within the asset's scope. If the strategy's performance on the unshuffled data is at the extreme of these paths, then it can be considered to be a significant result.

BRK-B

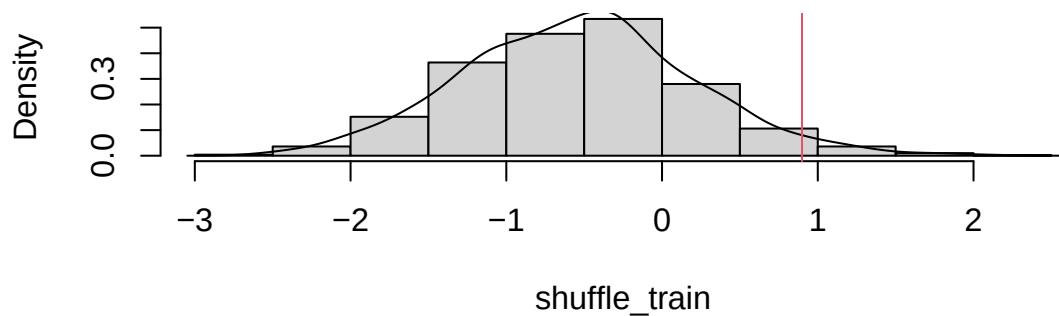
Performance on Training Data

```
shuffle <- \(data) {  
  xts::xts(  
    sample(as.numeric(data)),  
    order.by = index(data)  
  )  
}  
  
forecasting_model <- ge$best$expressions[1]  
train_signal <- with(TrainingData, eval(forecasting_model))  
train_close <- TrainingData[, close]  
next_day <- stats::lag(train_close, -1) |> log() |> diff() |> na.omit()  
train_return <- cumsum(train_signal * next_day)  
  
max_test <- 1000  
  
shuffle_train <- replicate(max_test, sum(train_signal * shuffle(next_day)))  
shuffle_mean <- mean(shuffle_train)  
shuffle_sd <- sd(shuffle_train)  
prob_ret <- pnorm(shuffle_mean, shuffle_sd, lower.tail = FALSE)
```

Training return vs buy-and-hold



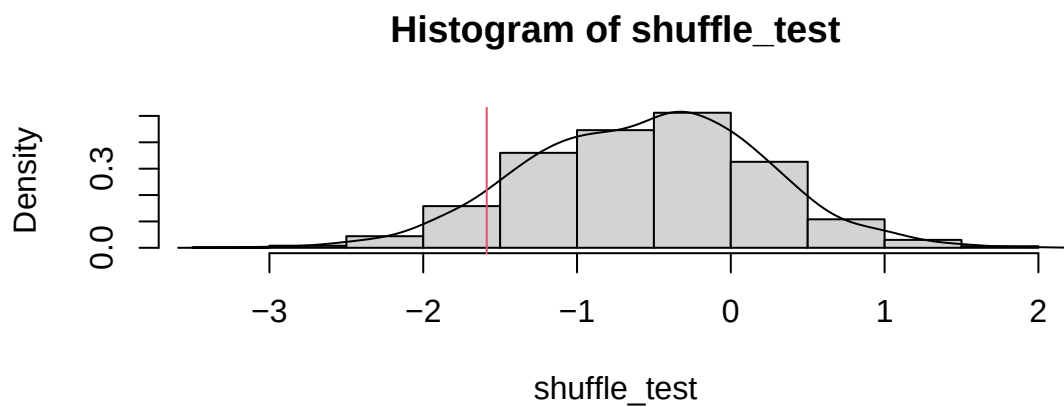
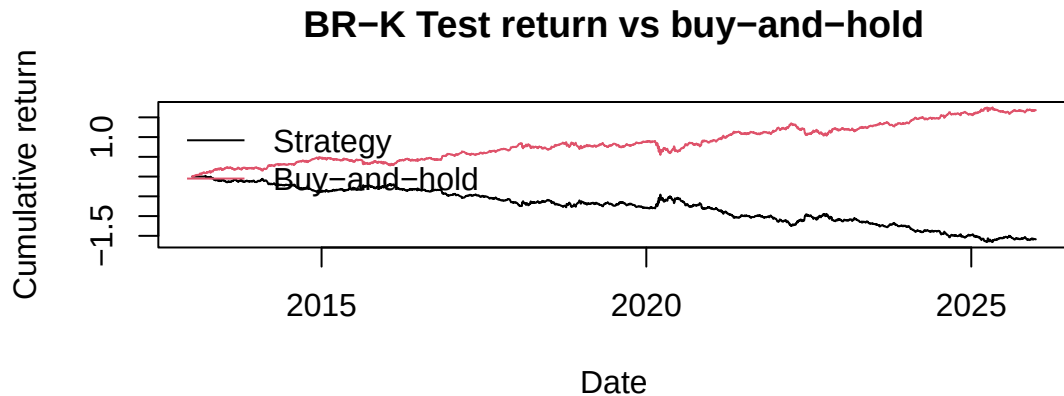
Histogram of shuffle_train



The GA found this model: `expression(comp(47L, cmo(BRK.B.High, 50L) + vwap(BRK.B.Low, BRK.B.Volume, 3L) - cmf(merge(BRK.B.High, BRK.B.Low, BRK.B.Close), BRK.B.Volume, 50L)))`

Performance on Test Data

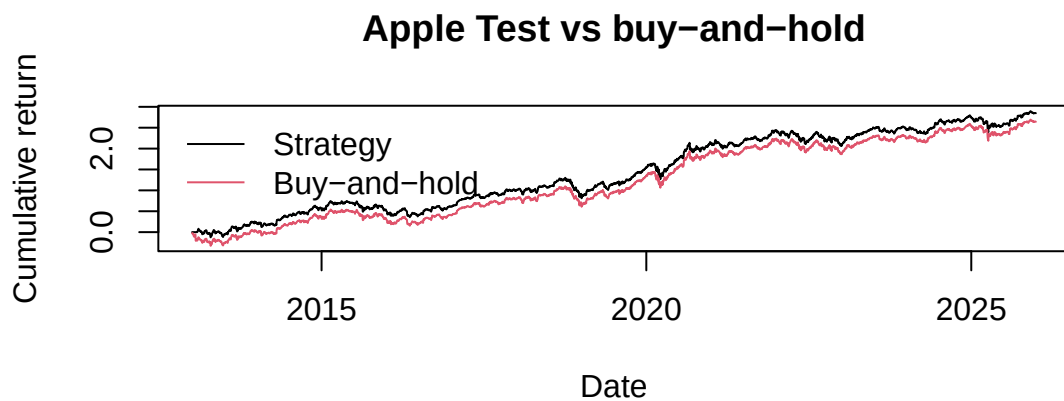
Applying the same strategy to the test period gave the following:



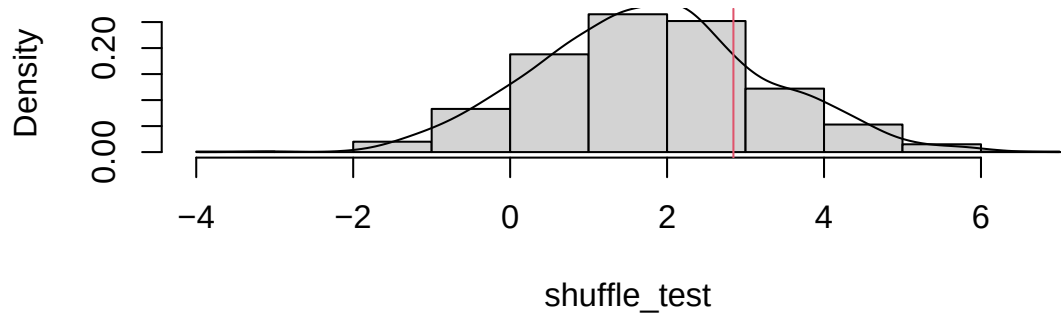
The results are obviously disappointing: a high-performing and reasonably significant strategy in the training set performed badly in the test set. Could it just be this asset? The approach was also tested against the other two assets, showing differing performance in each.

Apple

The GP found this model: `expression(comp(47L, cmo(BRK.B.High, 50L) + vwap(BRK.B.Low, BRK.B.Volume, 3L) - cmf(merge(BRK.B.High, BRK.B.Low, BRK.B.Close), BRK.B.Volume, 50L)))`



Histogram of shuffle_test

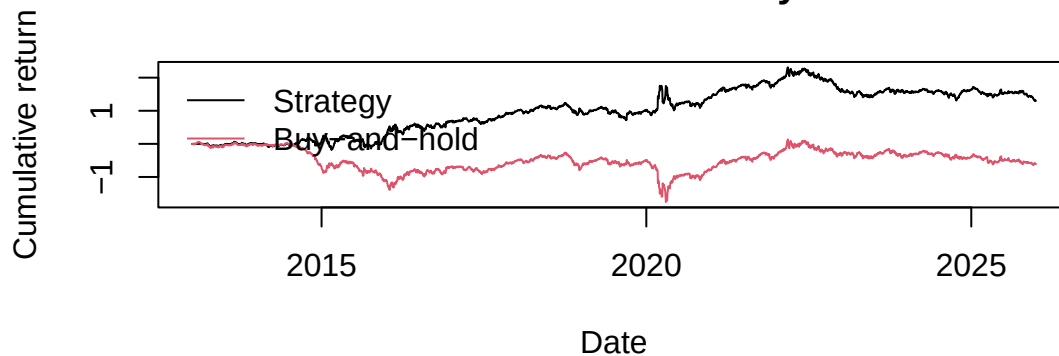


In this case, the GP has found a rather insignificant strategy that closely follows the buy and hold strategy in test. It compares a set of indicators that rely on momentum and volume to the number 47, which seems arbitrary.

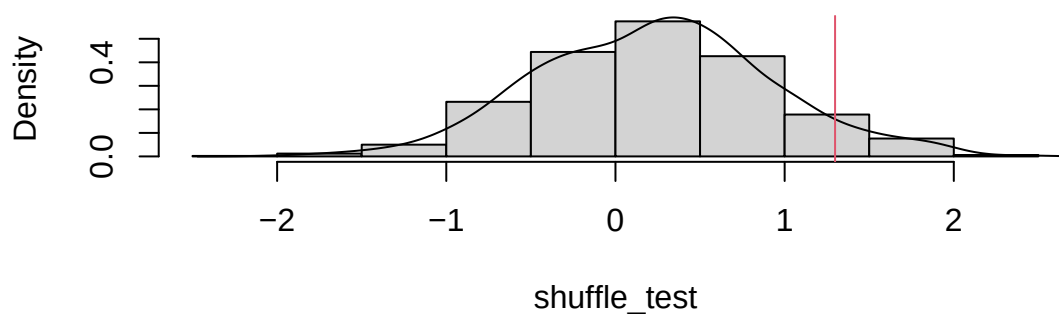
Brent Crude

Model: `expression(comp(BZ.F.Low, (vhf(BZ.F.Open, 3L) * BZ.F.Open - sma(BZ.F.Close, 5L)) * dema(BZ.F.High, 20L, 1, FALSE) * cmf(merge(BZ.F.High, BZ.F.Low, BZ.F.Close), BZ.F.Volume, 50L)))`

Brent-Crude Test return vs buy-and-hold



Shuffle Test



This strategy performs quite well compared to the asset in both test (not shown) and training. But the shuffle test's p -value of 0.6754511, is too low. We require a value above 0.95 for proper statistical significance. Nonetheless, part of the explanation for its more reliable performance could be that the underlying asset is more volatile than the others making buy and hold inherently unattractive.

Conclusion

The results are clearly lacklustre. For some assets our approach may work well, but in others the results are less impressive. Whilst developing this report, it was noted that there was a great deal of variation between different runs of the GA, suggesting that the random seed partly determined the result (which it shouldn't). On some occasions a very improbable, highly profitable strategy emerged. In others, it produced a loser. The implication is that the GA has not fully explored the problem space, which in turn suggests that a significantly higher number of iterations may be required, or a deeper grammar, or a higher `max.depth` setting, however this was not feasible with the current hardware and time constraints particularly given R's limited recursion capability.

Nonetheless, luck may have found the strategies, but it doesn't explain their performance. For example, the following highly successful, low probability strategy was found after 20,000 iterations (left overnight) on Brent-Crude:

```
dema(BZ.F.Open, 14L, 0.1, FALSE) + BZ.F.High - hma(BZ.F.Close, 3L)] >
zlema(BZ.F.Close, 100L)
```

DEMA, HMA, and ZLEMA are very fast, low-lag indicators, but with ZLEMA applied to a very long period. The result is essentially a moving average crossover indicator; a common strategy. Its behaviour is entirely deterministic; why should its performance be random?

In summary, this report indicates that technical indicators may have more predictive power than the Efficient Market Hypothesis would allow, but such an assertion requires more research on a larger universe of assets.

Self-Adjusting Algorithm

The following code was developed to simulate a trading system as part of the cost function, but was too slow to use in production. However, it could be extended to incrementally update the GA as it meets new data.

```
strategy <- initial_strategy

closed_trades <- list()
# Any strategy will require more than 1 data point, so taking the first one at
# this point isn't going to change anything but make accumulating the historical
# data with xts.rbind a lot easier
past_data <- data_set[1]
# Construct a trade object. The close_price should never be empty on a
# closed trade; close_price == NULL represents a program failure
mk_trade <- \(direction, open_price, fee) {
  list(direction = direction,
        open = open_price,
        close_price = NULL,
        fee = fee
  )
}

# This is a dummy starting position introduced to make computation easier below
open_position <- mk_trade(direction = 0, open_price = 0, fee = 0)

for (i in 2:NROW(data_set)) {
  #####UPDATE GA MODEL#####
  if ( modulus(i, six_months) == 0) {
    new_data <- past_data[six_months_ago:NROW(past_data)]
    ge2 <- GrammaticalEvolution(forecasting_grammar,
                               indicator_fit,
```

```

        suggestions = ge$best$genome # etc...)
strategy <- ge2$best$expresssions[1]
}
#####
# Any real-world strategy must generate a signal using yesterday's data
model <- tryCatch(
  with(past_data, eval(strategy)),
  error = function(e) {
    if (grepl("outside valid range", e$message)) {
      NULL
    } else { # have a fit
      stop(e)
    }
  }
)
# Signals on early data are likely to produce NA since the strategy will probably
# use moving averages. Skip these, but keep adding to past_data
latest_data <- data_set[i]
# Add today's ohlcv to the past_data series
past_data <- rbind.xts(past_data, latest_data)
# Get the last prices as a number
latest_close <- as.numeric(latest_data[, close])
latest_open <- as.numeric(latest_data[, open])
# Skip nulls from the tryCatch
if (is.null(model)) { next }
# Latest signal is the last signal in the model
signal <- as.numeric(last(model))
# Do some trading
current_direction <- open_position$direction
latest_close <- as.numeric(latest_data[, close])
if(current_direction != signal){
  #Close the current position
  open_position$close_price <- latest_close
  closed_trades <- c(closed_trades, list(open_position))
  # Open a new position, assuming a trading fee of 2% per trade.
  assumed_fee <- 0.02
  open_position <- mk_trade(direction = signal,
                             open_price = latest_close,
                             fee = assumed_fee
  )
}
# If we've reached the end of the dataset, just close the final trade
if(i == NROW(data_set)) {
  open_position$close_price <- latest_close
  closed_trades <- c(closed_trades, list(open_position))
}
}
trade_profit <- \(trade) (trade$direction * (trade$close_price - trade$open) - trade$fee)
total_profit <- lapply(closed_trades, trade_profit) |> as.numeric() |> sum()
print(total_profit)

```